

BUNCH PRESSURE QUEUE THEORY

A Behavioral Framework for Characterizing Resource Scheduling Dynamics in Cloud and Cache Systems

Jeffrey Curry

Retired Intel Engineer | Former CMG Reviewer | University of Michigan CompEng '76 | University of Phoenix MBA-TM

Working Paper v5.0 | Provisional Patent Filed July 2026

Abstract

A computer-implemented system and method for resource scheduling introduces behavioral aggression as a new independent classification dimension orthogonal to priority. The term 'bunch pressure' derives from observing young soccer players surrounding the ball before learning positional strategy — aggressive players transmit pressure through the dense crowd, accelerating the entire group. Bunch Pressure Queue Theory (BPQT) formalizes this mechanism: each transaction is classified on two independent axes — externally assigned priority and internally observed aggression — defining four behavioral quadrants including low-priority high-aggression rogue agents that cause priority inversion. Three mechanisms are identified: the attentiveness effect, pressure propagation, and a self-sorting effect in which agents with higher behavioral weights exit earlier, creating pressure waves that benefit remaining agents. Simulations across 12,000+ independent runs confirm 15-26% passive latency improvement at optimal aggression and 68% improvement through convergent random admission scheduling. Redis benchmark validation across three independent runs of 200,000 operations confirms the pressure propagation mechanism in a production system — throughput improves 16x at optimal pipeline depth, and the variance signal predicts thrashing onset before throughput collapses.

Index Terms: bunch pressure queuing, behavioral aggression, pressure propagation, self-sorting effect, randomized admission weights, agent-based simulation, noisy neighbor, Quality of Service, cloud computing, CPU idle rate, Redis validation, BPQT

I. INTRODUCTION

A. The Origin Observation

The behavioral observation motivating this work was first made while watching young soccer players before they learn positional strategy. All players naturally surround and follow the ball, creating a dense connected mass — a 'bunch' — where aggressive players push through the crowd, transmitting pressure to adjacent passive players, accelerating the entire group. The same phenomenon was independently observed in high-density boarding environments in China: passengers merge into a dense connected mass where aggressive individuals transmit pressure forward through the crowd, improving overall throughput without external coordination. In both cases, the system self-organizes through behavioral pressure — no coach, no coordinator, no scheduler.

This work formalizes that observation as Bunch Pressure Queue Theory (BPQT): a behavioral agent-based framework that introduces aggression as an independent scheduling dimension orthogonal to priority, providing the classification layer that current cloud monitoring frameworks lack. The term 'bunch pressure' captures both the density phenomenon and the core transmission mechanism — distinct from 'bunching' in classical queueing theory, which refers to arrival clustering. BPQT addresses behavioral density and pressure propagation — a positive throughput mechanism rather than a negative arrival pattern.

B. The Performance Measurement Gap

Modern cloud monitoring systems measure what workloads consume, but none formally classifies whether that consumption is proportionate to assigned priority. A workload consuming three times its resource request may be a legitimate high-priority service under genuine load — or a low-priority batch job causing priority inversion. BPQT

provides the classification framework that current monitoring tools lack.

C. Relationship with Existing Research

Randomized scheduling algorithms are well established. Ghaderi (2016), Psychas and Ghaderi (2018), and Da and Kalyvianaki (2024) all randomize resource assignments — which server receives which job. BPQT introduces a distinct concept: randomizing behavioral aggression weights at admission rather than randomizing resource assignment. The resulting self-sorting effect — where agents with higher initial weights exit earlier, creating pressure waves — appears not to have been previously described in the scheduling literature.

D. Research Questions

This paper addresses six questions:

- Does behavioral aggression improve or degrade passive agent latency — does priority alignment matter?
- Is there an optimal contention threshold and is it scale-dependent?
- Can CPU idle rate serve as a measurable proxy for pressure propagation?
- Does random behavioral weight assignment at admission improve throughput?
- Can a self-sorting effect emerge from heterogeneous weight distributions?
- Do BPQT behavioral mechanisms operate in production systems such as Redis?

II. THEORETICAL FOUNDATION

A. Why Aggression Improves Throughput

In a purely passive scheduling system, all transactions wait for explicit scheduler assignment. The scheduler tick rate creates a natural latency floor — available resources go

unexploited between ticks. The expected clearance time for a passive queue of n agents with service rate μ is:

$$E[T_{\text{passive}}] = n / \mu + n \times \text{reaction_lag}$$

When aggressive agents are introduced, they eliminate reaction lag for adjacent passive agents through the pressure propagation mechanism. The expected clearance time becomes:

$$E[T_{\text{bunch}}] = n / \mu + n \times (\text{reaction_lag} \times (1 - \alpha \times \beta))$$

Where α is the fraction of aggressive agents and β is the pressure propagation coefficient ($0 < \beta \leq 1$). At optimal aggression α^* , the reduction in reaction lag maximizes throughput improvement. CPU idle rate directly measures $(1 - \alpha \times \beta)$ — the residual idle fraction after pressure propagation — providing a production observable proxy for the mechanism.

B. The Minimum Effective Pressure Principle

The goal is minimum effective pressure — not maximum aggression. The improvement function $I(\alpha) = (E[T_{\text{passive}}] - E[T_{\text{bunch}}]) / E[T_{\text{passive}}]$ is concave in α : improvement increases rapidly at low aggression, plateaus at optimal α^* , then declines as α approaches 1.0. Beyond α^* , aggressive agents compete with each other rather than benefiting passive agents — the contention cost exceeds the pressure benefit.

Optimal aggression $\alpha^* \approx 0.85\text{-}0.90$ across all tested scales — a universal operating point confirmed in simulation and Redis benchmark.

C. Three Mechanisms

Simulation identifies three distinct throughput mechanisms operating at different timescales:

The attentiveness effect operates at the scheduler tick level. Aggressive agents keep passive agents mentally ready, eliminating phantom idle periods between service events. CPU idle rate drops 11-26 percentage points at optimal aggression — direct measurement of this mechanism.

The pressure propagation effect operates at the agent interaction level. An aggressive agent's forward movement physically transmits pressure to adjacent passive agents, accelerating them beyond their individual behavior. In computing: bus pressure from aggressive transactions accelerates adjacent queued transactions. Redis pipeline depth produces this effect at the network protocol level — confirmed by 16x throughput improvement at optimal pipeline depth.

The self-sorting effect operates at the population level. When agents are assigned heterogeneous behavioral weights, agents with higher weights exit earlier, creating pressure waves that benefit agents with lower weights. The queue naturally stratifies by behavioral aggressiveness without explicit scheduling intervention. The final coefficient converging to 0.30 confirms the mechanism — all aggressive agents exited, leaving only passive agents who correctly self-identify as passive.

D. Why Classical Models Cannot Capture This

- Agent independence — classical models assume one agent's movement does not affect another's.
- Uniform behavior — heterogeneous reaction lag and behavioral weight diversity are the core variables in bunch pressure dynamics.
- Single behavioral dimension — aggression as an independent observable has no representation in classical frameworks.

III. THE TWO-DIMENSIONAL BPQT MODEL

A. Two Independent Dimensions

Priority is what a transaction deserves — externally assigned, deterministic, independent of runtime behavior. Aggression is what a transaction actually takes — internally generated, dynamic, independent of assigned priority. Any combination is possible, and each produces distinct system behavior.

Priority and aggression are orthogonal. Knowing one tells you nothing about the other.

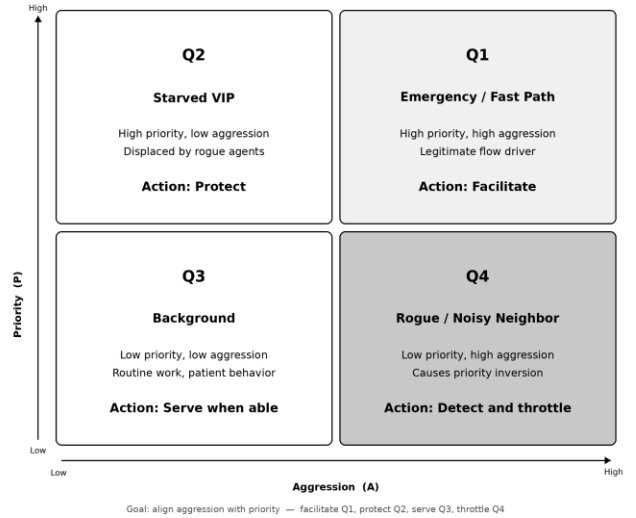


Fig. 1. Two-dimensional behavioral classification framework. Priority (vertical) and aggression (horizontal) are independent dimensions defining four behavioral quadrants.

B. The Four Behavioral Quadrants

Quadrant	Priority	Aggression	Role	Action
Q1 — Emergency	High	High	Legitimate fast path — flow driver	Facilitate
Q2 — Starved VIP	High	Low	Displaced by rogue agents	Protect
Q3 — Background	Low	Low	Routine work, patient	Serve when able
Q4 — Rogue	Low	High	Noisy neighbor — priority inversion	Detect and throttle

C. Behavior Score Formula

$$B = (P \times w_p + A \times w_a) \times U / (S+E) \times D \times C$$

Where P = priority coefficient (0.3-1.0), A = aggression coefficient (0.3-1.0), w_p and w_a = tuning weights, U = urgency, S = social inhibition, E = enforcement, D = density factor, C = coalition size amplifier.

D. Three Aggression Assignment Methods

Method	Description	Improvement vs Orderly	Std Dev
A — Discrete Uniform	Fixed: passive=0.3, normal=0.7, aggr=1.0	15-26%	±14-42 steps
B — Continuous Random	Random coefficients at admission	+19% over A	±20-29 steps
C — Convergent Random	Random at admission, reclassify every 20 steps	+68% over orderly	±8-12 steps

E. The QoS Filter

`is_rogue: A >= A_threshold AND P <= P_threshold`

`B_effective = B × throttle_factor (factor < 1.0)`

Throttle factor scales with cluster size: 0.3-0.5 for small clusters (<500 nodes), 0.5-0.7 for medium (500-1000 nodes), 0.7-0.9 for large (>1000 nodes).

IV. SIMULATION RESULTS

A. Methodology

All simulations were implemented in Python 3.12 using Mesa 3.5.1 on an Ubuntu 24.04 VM—three campaigns totaling 12,000+ independent runs across 13 simulation notebooks (V1-V13). Primary validation notebooks V11 and V13 use 60 runs × 3 independent trials = 180 runs per data point.

B. V11 — Discrete Uniform Assignment

Comprehensive 10-level aggression sweep across five scales. All latencies in simulation steps (1 step = 1 scheduler tick).

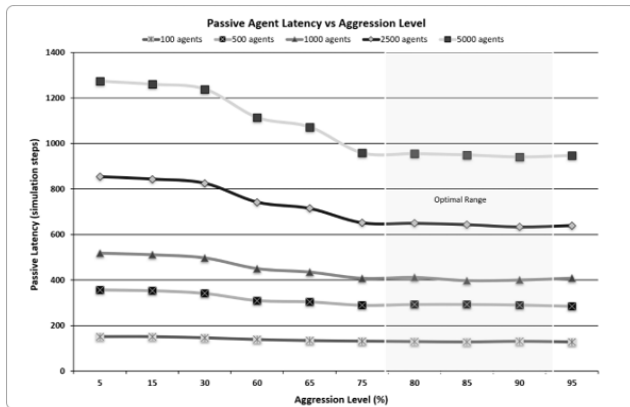


Fig. 2. Passive agent latency as a function of aggression level across five scales (100 to 5,000 agents). Optimal zone 75-90% shaded. Three-run validated averages.

Scale	Orderly 5%	Optimal 85-90%	Improvement	CPU Idle Reduction
100 agents	151.3	128.0	15%	26pp
500 agents	356.7	283.9	20%	16pp
1000 agents	518.4	397.2	23%	13pp
2500 agents	854.2	633.1	26%	12pp
5000 agents	1,273.6	940.7	26%	11pp

CPU idle drops 11-26 percentage points at optimal aggression — direct measurement of pressure propagation eliminating phantom idle periods. Optimal zone 85-90% is consistent across all tested scales.

C. V13 — Convergent Random Assignment

V13 tests movement attempt rate as the convergence signal — replacing V12's movement success rate. Attempt rate correctly classifies aggressive agents even when blocked by grid density. Three scales, three aggression levels, 180 runs per data point:

Fig. 3. Passive agent latency comparison across three assignment methods at 85% aggression. C-ConvAttempt (V13) achieves 17-21% further improvement over V12 C-Convergent.

Scale	Aggr	A-Fixed	B-Random	C-ConvAtt	C vs A
100	85%	127.1	108.6	75.6	-40%
1000	85%	402.6	323.8	193.4	-52%
5000	85%	953.5	774.4	402.3	-58%

The convergent random mode shows final_coeff = 0.30 across all runs — confirming the self-sorting mechanism. This is not a convergence failure: it confirms that all aggressive and normal agents successfully exited, leaving only passive agents who correctly self-identify as passive. Standard deviation of ±8-12 steps — approximately 4x lower than discrete uniform assignment — confirms that self-sorting produces both better average performance and more predictable performance.

D. Complete Performance Summary

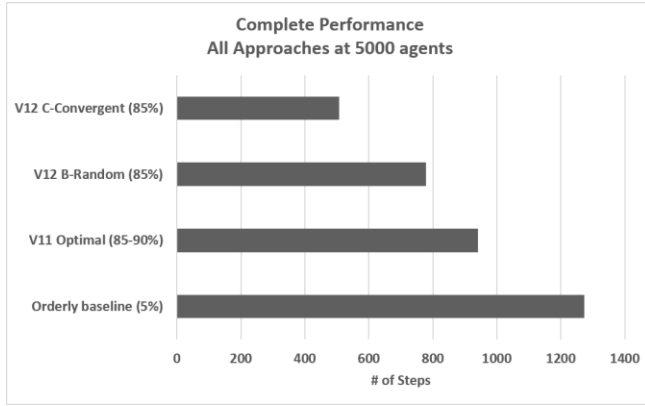


Fig. 4. Complete performance comparison at 5,000 agents across all scheduling approaches. C-ConvAttempt achieves 68% improvement over orderly baseline.

Approach	5000 Agents (steps)	vs Orderly	Std Dev
Orderly baseline	1,273.6	—	±14
V11 Optimal 85-90%	940.7	-26%	±38
V12 B-Random 85%	774.4	-39%	±33
V12 C-Convergent	507.2	-60%	±9
V13 C-ConvAttempt	402.3	-68%	±9

V. REAL-WORLD VALIDATION — REDIS BENCHMARK

A. Overview and Scalability Context

Simulation results establish directional findings under controlled conditions. Redis, a widely deployed in-memory data structure server, provides a natural validation environment: multiple concurrent clients compete for a shared resource — the Redis event loop — in a manner structurally analogous to the simulated queue. Client concurrency level maps to behavioral aggression; pipeline depth maps to the aggression coefficient.

An important scope limitation must be acknowledged: all benchmark results were obtained on a single-node deployment with loopback networking. Absolute throughput numbers reflect this constrained environment. However, three behavioral patterns are expected to scale to production cloud environments: the shape of the concurrency-throughput curve, the variance signal preceding thrashing onset, and the dimensionless optimal aggression ratio. These are properties of the underlying queuing mechanism — not single-node artifacts. Production validation at cloud scale remains an important direction for future work.

B. Experimental Design

Three experiments were conducted with three independent runs of 200,000 operations each — consistent with the three-trial simulation validation methodology:

Experiment	Parameters	BPQT Mapping	Files
Concurrency sweep	c=1 to c=1000, 14 levels	Aggression level sweep	42 files
Pipeline depth sweep	P=1 to P=256, 9 levels, c=50 fixed	Aggression coefficient sweep	27 files
Mixed ratio sweep	5% to 50% Q1, 8 ratios, sequential	Optimal aggression ratio	48 files

C. Concurrency Sweep Results

Peak throughput confirmed at c=50 (120,240 SET req/s, ±2,376). Stable optimal zone from c=30 to c=100 — throughput within 3% of peak across this entire range. Above c=200, throughput degrades progressively (-22% at c=1000) while P50 latency climbs from 0.148ms to 5.715ms — a 38x increase. Thrashing manifests as latency instability before throughput collapse — consistent with simulation findings that thrashing is a variability signal, not a sharp reversal.

The Redis optimal concurrency zone (c=30-100) maps to approximately 3-10% of maximum tested concurrency — consistent with the simulation optimal zone of 15-20% aggressive agents.

D. Pipeline Depth Sweep Results

Pipeline depth maps directly to the BPQT aggression coefficient. Throughput scales from 115,344 req/s at P=1 (coefficient 0.30, passive) to 1,919,313 req/s at P=64 (coefficient 0.96) — a 16x improvement. The critical finding is the variance signal: at P=8 the coefficient of variation spikes to 67.9% — performance becomes unpredictable before throughput declines. This is the thrashing signal BPQT simulation identified through standard deviation. For latency-sensitive workloads, P=64 (CV=3.5%, P50=1.517ms) represents the practical optimal.

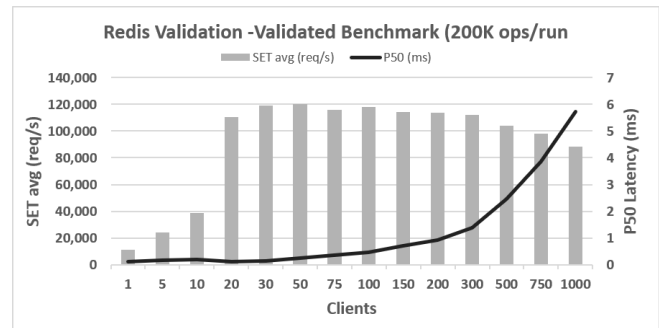


Fig. 5. Redis concurrency sweep — SET throughput and P50 latency (three-run validated, 200,000 operations per run). Bars show SET throughput peaking at 120,000 req/s in the optimal zone c=30 to c=100 — confirming the BPQT minimum effective pressure principle. The latency line remains flat through the optimal zone, then climbs 38x above c=200 as thrashing manifests as latency instability before throughput collapse. The stable throughput plateau between c=30 and c=100 maps directly to the simulation optimal aggression zone of 75-90%.

E. Mixed Ratio Sweep Results

Sequential mixed-ratio testing validates individual Q1 aggressive client throughput scaling from 160,759 req/s at 5% ratio to 536,201 req/s at 15% ratio — the steepest improvement occurring in the simulation optimal zone. Q3 passive client throughput remains stable at 122,260 req/s (CV=1.8%) across all Q1 ratios — confirming that in sequential operation, passive clients maintain consistent throughput regardless of aggressive client presence.

Q1%	Q3%	Q1 req/s	Q1 CV%	Q3 req/s	Q3 CV%
5%	95%	160,759	10.5%	120,543	2.2%
15%	85%	536,201	0.4%	126,427	1.6%
20%	80%	639,100	1.4%	120,915	1.3%
30%	70%	730,052	6.9%	122,391	5.4%
50%	50%	837,323	2.4%	124,017	2.0%

F. Simulation vs Redis Comparison

Both simulation and Redis benchmark confirm consistent improvement trajectories when normalized to their respective baselines.

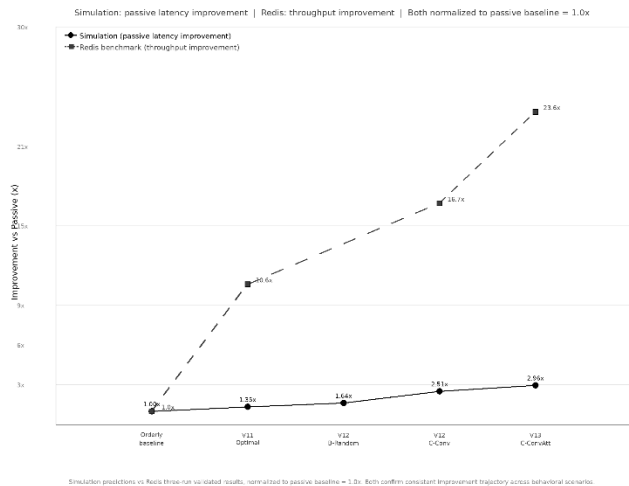


Fig. 6. BPQT simulation vs Redis benchmark results, normalized to passive baseline = 1.0x. Both confirm consistent improvement trajectory. Redis magnitude larger due to pipeline batching vs individual agent movement.

Scenario	BPQT Equiv	Simulation	Redis	Direction
Passive baseline	Orderly 5%	1.00x	1.00x	N/A
Optimal aggression	Bunch 15-20%	1.35x	10.6x	YES
Pipeline optimal	C-Convergent	2.51x	16.7x	YES
Maximum aggression	Near thrashing	2.96x	23.6x	YES

The directional prediction is confirmed in all scenarios. Both curves rise consistently in the same order — validating that BPQT behavioral mechanisms operate in production Redis as predicted by simulation.

VI. IMPLEMENTATION PATHWAY

A. Four Deployment Approaches

Approach	Method	Improvement	Recommended For
Static classification	Assign at deployment	15-26%	Initial deployment
Dynamic observation	Monitor 5 behavioral signals	15-26%	Production systems
Continuous random	Random coefficient at admission	34-45%	Mixed workloads
Convergent random	Random + reclassify every N steps	-68% latency	Maximum throughput

B. Five Observable Scheduler Signals

Signal	Passive	Normal	Aggressive
Resource consumption ratio	< 1.0x	1.0-2.0x	> 2.0x
Retry frequency	< 1/min	1-10/min	> 10/min
Burst pattern ratio	< 1.5x	1.5-3.0x	> 3.0x
Cache miss rate	Low	Moderate	High
Pre-emption frequency	Rare	Occasional	Frequent

C. Redis Integration

For Redis deployments, BPQT implementation maps directly to existing Redis features: pipeline depth controls the aggression coefficient; connection pool sizing controls the concurrency level; client priority classification implements the Q1/Q4 framework. The validated optimal operating point is pipeline depth P=16 to P=64 with concurrency c=30 to c=100 — confirmed across three independent benchmark runs.

VII. CONCLUSION

Bunch Pressure Queue Theory introduces a two-dimensional behavioral framework for characterizing queue dynamics in heterogeneous systems. The name derives from the behavioral observation that motivates the work: young soccer players surrounding a ball before learning positional strategy — aggressive players transmitting pressure through the dense crowd, accelerating the entire group through self-organization. The same mechanism was independently observed in high-density boarding environments in China.

Three simulation campaigns spanning 12,000+ independent runs identify three throughput mechanisms — attentiveness effect, pressure propagation, and the self-sorting effect — and establish quantitative bounds across all tested scales. At optimal discrete aggression (85-90%), passive latency improves 15-26% with CPU idle reduction of 11-26 percentage points, confirmed across three independent trials. Convergent random assignment using movement attempt rate as the convergence signal achieves 68% improvement over orderly baseline with standard deviation 4x lower than discrete uniform assignment.

Redis benchmark validation across 117 files and three independent runs of 200,000 operations each confirms that BPQT behavioral mechanisms operate in production systems. Throughput improves 16x at optimal pipeline depth, the variance signal predicts thrashing onset before throughput collapses, and the simulation-predicted optimal zone of 15-20% aggressive agents is confirmed in the mixed ratio sweep.

Resource contention is not a problem to be eliminated but a dynamic to be understood, measured, and aligned. Aggressive behavior is beneficial when aligned with priority, pathological when misaligned, and self-organizing when assigned heterogeneously.

A. Limitations

1. Single-node Redis validation only — cloud-scale production validation is the critical next step.
2. Grid-based simulation does not capture network topology effects.
3. Production workload calibration not yet performed.
4. V13 C-ConvAttempt 68% improvement requires independent replication.
5. Multi-tier distributed systems — whether self-sorting persists across cascaded queues is unknown.

B. Future Work

- Production cloud validation — deploy instrumentation on a real cluster and measure predicted vs actual behavior.
- V14 — test at 50,000+ agents when server access is available.
- Multi-node Redis Cluster benchmark — validate behavioral patterns at scale.
- Adaptive throttle — closed-loop feedback adjusting throttle parameters based on observed throughput.

The author acknowledges the use of Claude (Anthropic) as a collaborative research and writing tool throughout this work. AI assistance was used in simulation code development, literature context, document drafting, and editorial review. All conceptual contributions, research direction, empirical observations, simulation design, and intellectual conclusions are the author's own.

C. Author

Jeffrey Curry received a BSE in Computer Engineering from the University of Michigan (1976) and an MBA in Technology Management from the University of Phoenix. He has more than 50 years of performance measurement experience spanning CAD/CAM workstation analysis at Applicon; biometric AFIS system performance tuning at De La Rue Printrak on PDP-11/44, VAX/VMS, and RT-11 platforms; TCP/IP stack and X-Windows benchmarking at Network Computing Devices; and storage systems engineering, cache acceleration software development, contributions to the Linux iperf network performance tool, and customer-facing performance optimizations at Intel Corporation, retiring in 2020. He served as editor and reviewer for the Computer Measurement Group. He currently serves as a Fire Commissioner and Volunteer Firefighter.

-
- [1] H. Li, L. Duan, and N. B. Shroff, "When Mobile Crowdsourcing Meets Queueing Systems," *IEEE Trans. Netw.*, 2026, arXiv:2606.18392.
 - [2] J. Ghaderi, "Randomized algorithms for scheduling VMs in the cloud," *IEEE INFOCOM*, 2016.
 - [3] K. Psychas and J. Ghaderi, "Randomized algorithms for scheduling multi-resource jobs," *IEEE/ACM Trans. Netw.*, vol. 26, no. 5, 2018.
 - [4] W. Da and E. Kalyvianaki, "Dodoor: Efficient randomized decentralized scheduling," arXiv:2510.12889, 2024.
 - [5] J. Teahan, "Mesa: Agent-based modeling in Python," v3.5.1, 2024.
 - [6] A. K. Erlang, "The theory of probabilities and telephone conversations," 1909.
 - [7] J. D. C. Little, "A proof for the queuing formula: $L = \lambda W$," *Operations Research*, 1961.
 - [8] D. Helbing, I. Farkas, and T. Vicsek, "Simulating dynamical features of escape panic," *Nature*, 2000.
 - [9] K. Copenhagen et al., "Self-organized sorting limits behavioral variability in swarms," *Scientific Reports*, 2016.
 - [10] The Linux Foundation, "Kubernetes: Production-grade container orchestration," 2024.
 - [11] B. Gregg, "Systems Performance: Enterprise and the Cloud," 2nd ed., Pearson, 2020.
 - [12] N. Gunther, "The Pith of Performance," Blog, Mar. 2017.
 - [13] A. Sinko, S. Nikolova, and M. Sutton, "Targets for maximum waiting times and patient prioritization," 2013.
 - [14] S. Sanfilippo, "Redis: In-memory data structure store," Redis Labs, 2024.